Homework #3

Yousef Awad

October 18, 2026

Questions 1 & 2: Fault Detection and Correction Methods

For $C = AB$ (MMM, $A, B \in \mathbb{R}^{n\times n}$) and $y = Ax$ (MVM, $A \in \mathbb{R}^{m\times n}$). "Memory" covers weights/activations at rest in DRAM/HBM/SRAM; "Compute" covers the multiply-accumulate (MAC) datapath.

Table 1: Fault Detection Methods

Method	Focus	Advantages	Disadvantages
Parity	Memory	1 bit/word; trivial logic	Odd-bit errors only
CRC	Memory / link	Strong burst detection; cheap encoder	Needs retry or companion ECC
SEC-DED (Hamming)	Memory	Detects 2-bit errors; low latency	12.5% storage; misses chip faults
Residue code (mod A)	Compute	Small checker; preserved under $+$, $\times$	Zero-residue blind spot
AN code	Compute	Checks arithmetic directly	Re-encoding; $AN \times AN$ needs $1/A$ fix
Range assertions / activation statistics	Mem / Compute	Near-zero cost; catches NaN/Inf, large flips	Misses small in-range errors
ABFT sum checksum (Huang–Abraham)	Compute (+ memory)	$O(n^2)$ check for $O(n^3)$ MMM; offline checksums also catch bad weights	FP tolerance needed; MVM checksum cannot locate
Dual Modular Redundancy (spatial)	Compute	Catches any single-copy fault	100% area; can't tell which copy failed
Re-execution / RESO (temporal)	Compute	No extra hardware; RESO finds permanent faults	$\geq 2\times$ latency
BIST / March tests	Memory	Finds hard faults for sparing	Offline only; misses transients

Table 2: Fault Correction Methods

Method	Focus	Advantages	Disadvantages
SEC-DED (Hamming)	Memory	Inline 1-bit correction	Fails on multi-bit/chip faults
Reed–Solomon / Chipkill	Memory	Survives whole-chip failure	Decoder latency and area
Decoupled detect-then-correct	Memory	Fast common path (VECC, HBM-CASO)	Slow path on error; extra metadata
Memory scrubbing	Memory	Stops soft-error accumulation	Bandwidth/power; no hard-fault fix
Row/column/chip sparing	Memory	Permanently repairs hard faults	Limited spares; needs BIST
RRNS (redundant residue number system)	Compute	Corrects arithmetic errors; carry-free	RNS conversion cost
Full-checksum ABFT (MMM)	Compute	Row/column syndromes locate and fix one error	One error per row/column
Weighted two-checksum ABFT (MVM)	Compute	Location $= \Delta_1/\Delta_0$, magnitude $= \Delta_0$	Weights amplify FP rounding
TMR / NMR	Compute	Masking with no recovery latency	>200% area/power; voter is a single point of failure
Re-execution on detection	Compute	Cheap; fixes transients	Permanent faults recur
Approximate correction (zero-out / clipping)	Compute	Near-zero cost; stops error explosion	Inexact
Checkpoint / rollback	Compute	Handles bursts beyond ABFT	Storage; variable recovery delay

Question 3: Integrated Protection Strategy & Rationale

My Proposed Protection Strategy: Use 3 tiers, with cheap detection first and then correction only whenever detection finds an error.

- **Tier 1 (Memory):** RS/Chipkill ECC on DRAM/HBM (detect on every read, decode only on error), SEC-DED on SRAM buffers, plus scrubbing and BIST-guided sparing for hard faults.
- **Tier 2 (Compute):** ABFT with weight checksums precomputed offline. Full checksum for MMM (row/column syndromes locate and fix the error), weighted two-checksum for MVM (location = Δ_1/Δ_0). Mod-3 residue checks on bias/scaling ops that ABFT doesn't cover.
- **Tier 3 (Fallback):** If ABFT can't correct it, re-execute the tile; if the error comes back, treat it as permanent and remap to a spare PE. Zero-out only in layers that profiling shows are error-tolerant.

Design Rationale: TMR costs way too much with it costing >200% area/power and sadly ECC alone leaves the MAC datapath exposed, so if we add on an Algorithm-Based Fault Tolerance method (ABFT) it can do the heavy lifting. As well it only adds $O(n^2)$ extra work for $O(n^3)$ MMM (at least according to my research using Gemini Notebook and trying to understand it more). MVM is however memory-bound (each weight used once), and so with it, Tier 1 matters most. Thankfully, since faults are rare every tier only pays for a correction whenever something is actually detected. The tiers also help back each other up: the offline weight checksums catch memory errors that get past ECC, and ECC keeps ABFT from seeing more errors than it can correct.